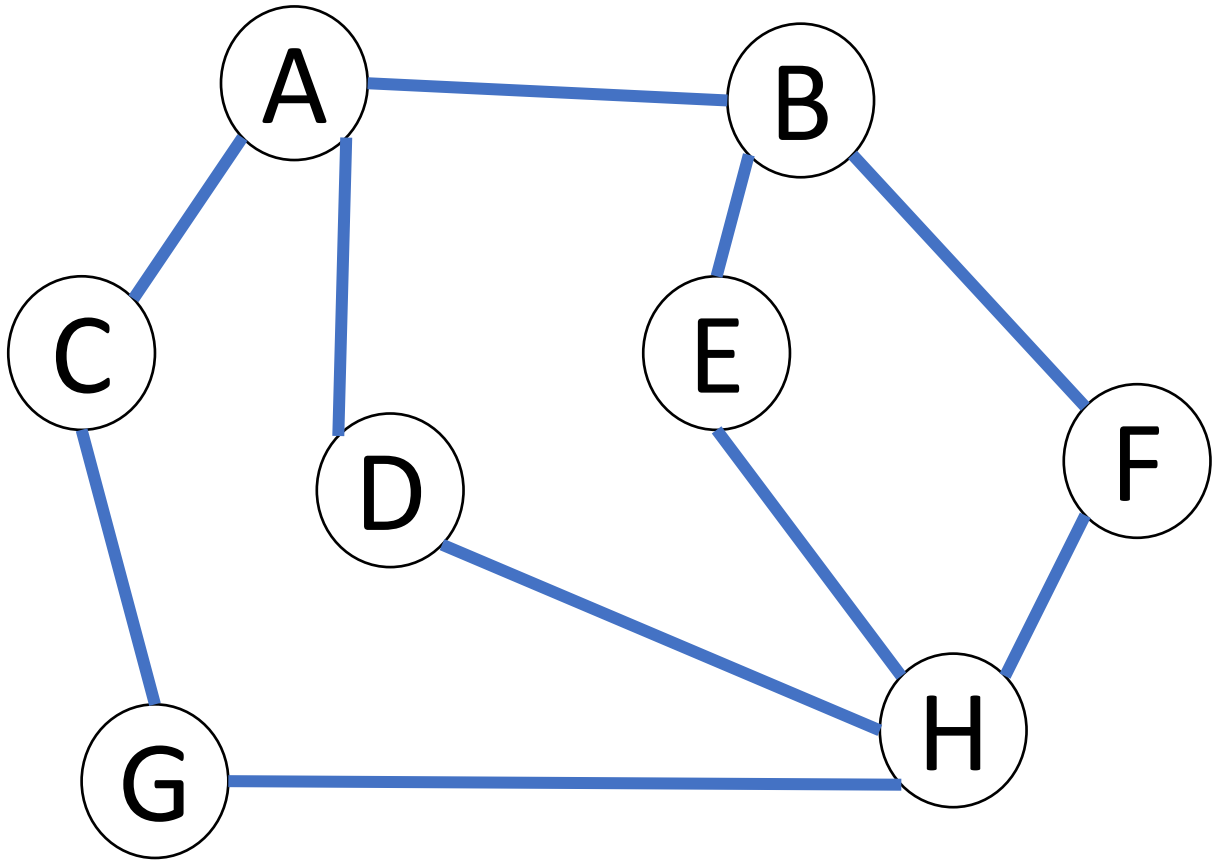
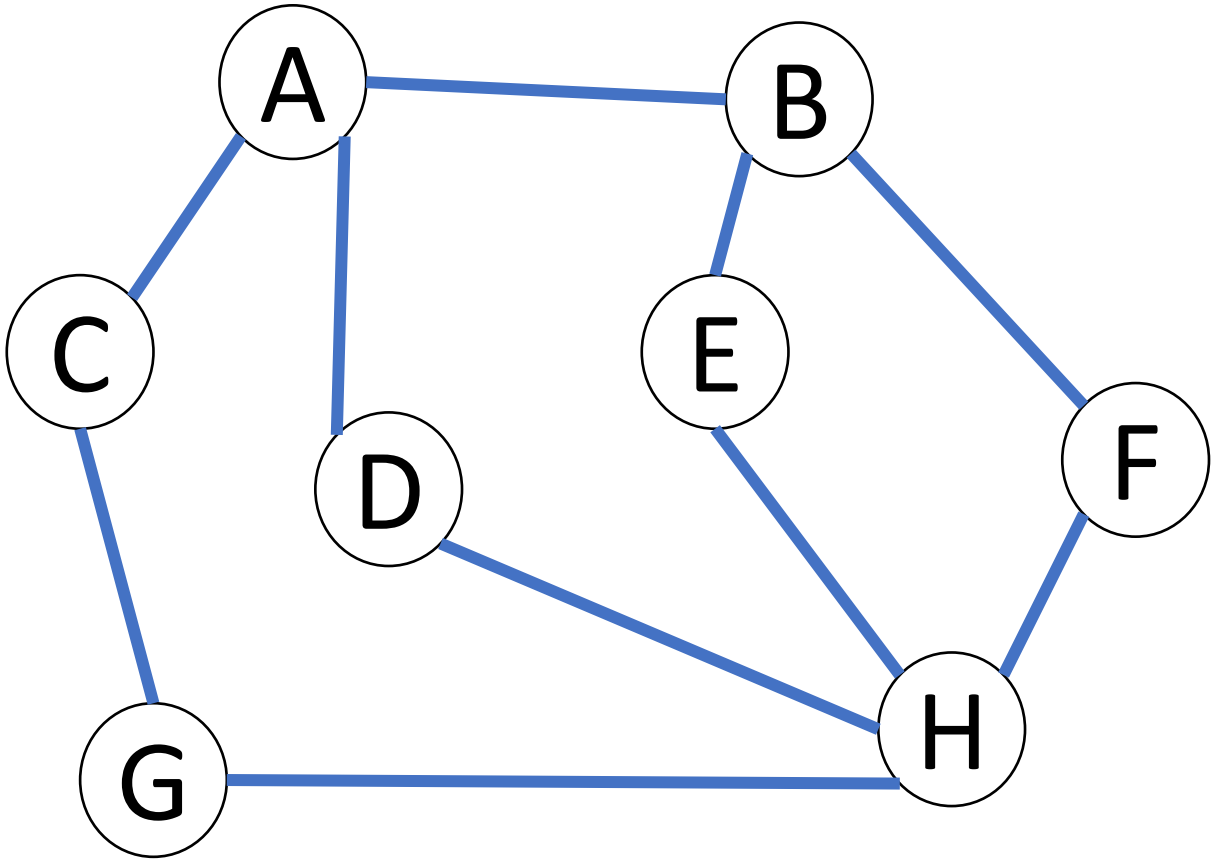
- $G = (V, E)$



Vertex List

A
B
C
D
E
F
G
H

Edge List

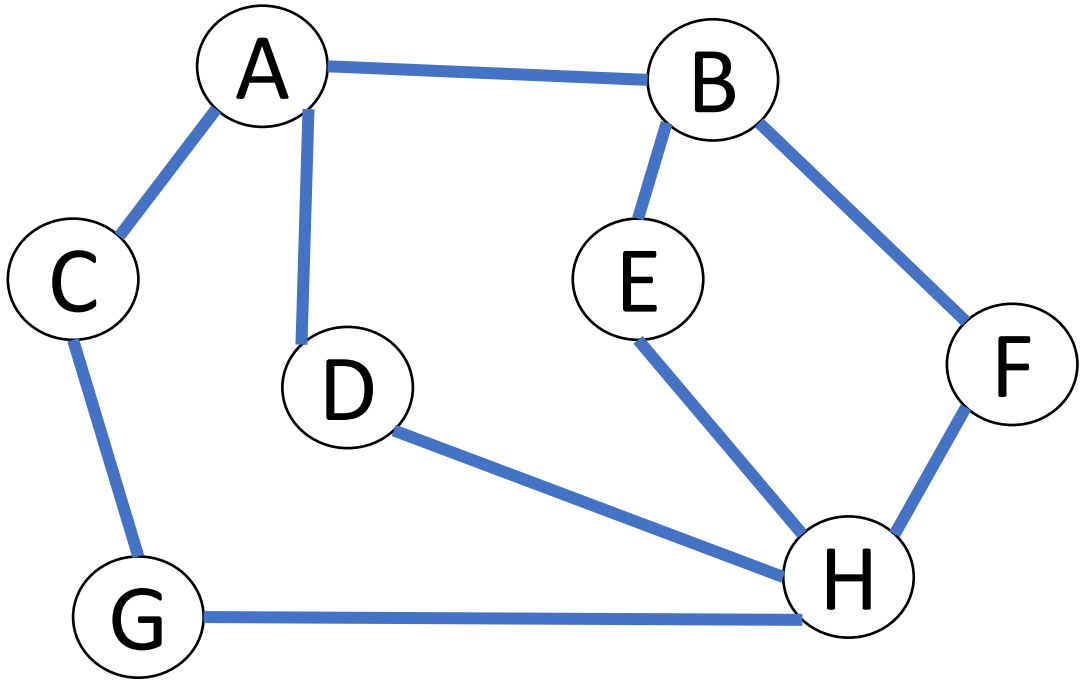


Vertex List

A
B
C
D
E
F
G
H

Edge List

A	B
A	C
A	D
B	E
B	F
C	G
D	H
E	H
F	H
G	H




```
Struct  Edge
{
        char *startVertex;
        char *endVertex;
};
```

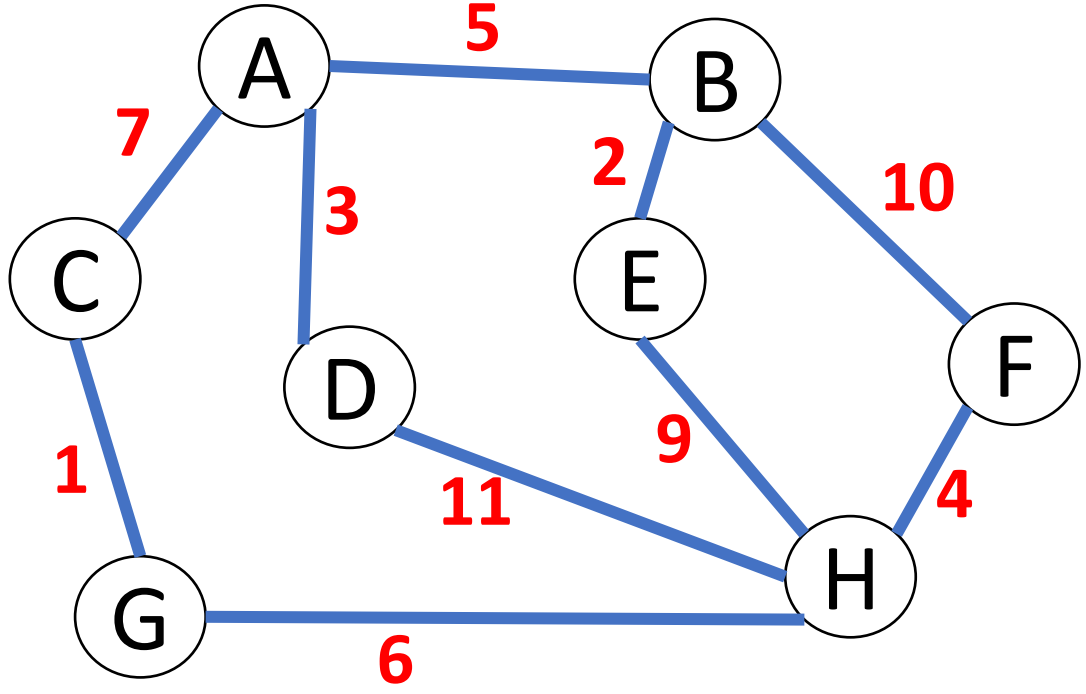
```
Class  Edge
{
        string startVertex;
        string endVertex;
};
```

Vertex List

A
B
C
D
E
F
G
H

Edge List

A	B	5
A	C	7
A	D	3
B	E	2
B	F	10
C	G	1
D	H	11
E	H	9
F	H	4
G	H	6




```
Struct  Edge
{
        char *startVertex;
        char *endVertex;
        int weight;
};
```

```
Class  Edge
{
        string startVertex;
        string endVertex;
        int weight;
};
```

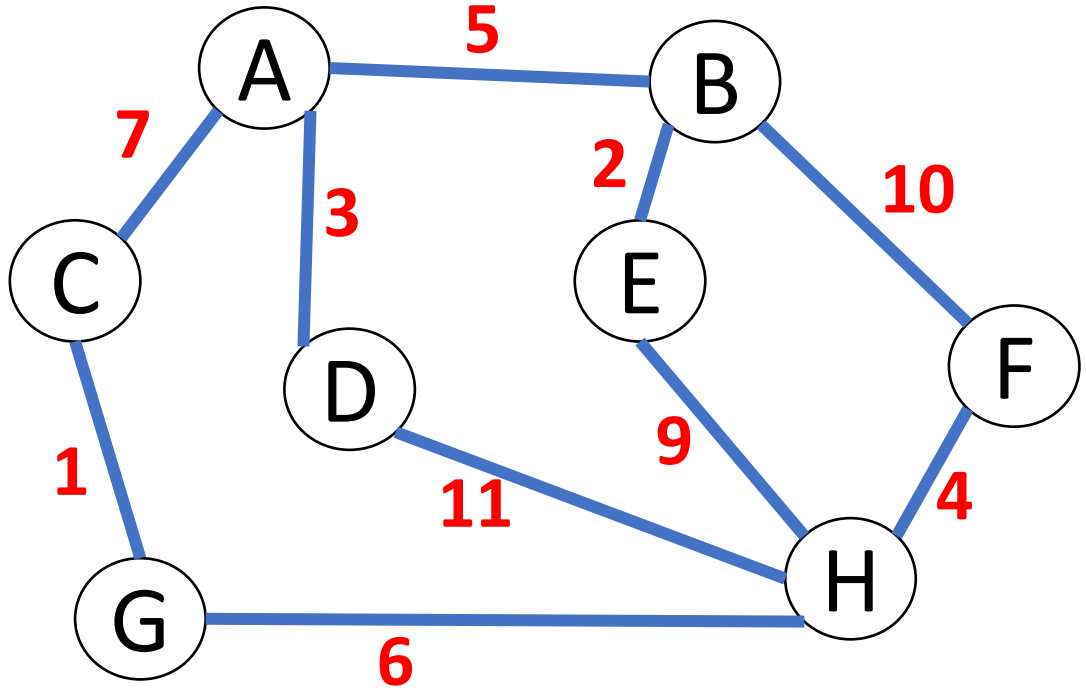
```
string vertex_list[MAX_SIZE];
Edge  edge_list[MAX_SIZE];
```

Vertex List

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

Edge List

0	1	5
0	2	7
0	3	3
1	4	2
1	5	10
2	6	1
3	7	11
4	7	9
5	7	4
6	7	6




```
Struct  Edge
{
        int *startVertex;
        int *endVertex;
        int weight;
};
```

```
Class  Edge
{
        int startVertex;
        int endVertex;
        int weight;
};
```

```
string vertex_list[MAX_SIZE];
Edge  edge_list[MAX_SIZE];
```

Space & Time Complexity

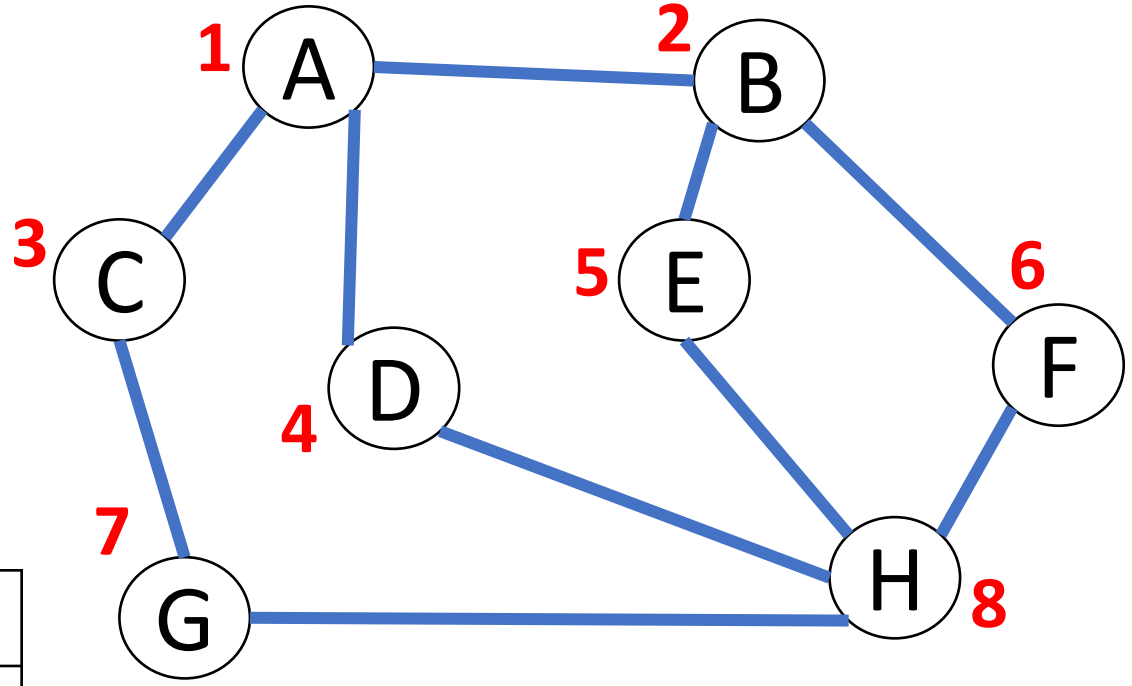
- Memory usage (Space complexity)
  - $\boldsymbol{O}(|V|)$
  - $\boldsymbol{O}(|E|)$
  - $\boldsymbol{O}(|V| + |E|)$
- Time cost of operations (Time complexity)
  - Find all adjacent nodes
    - Running time: $\boldsymbol{O}(|E|)$
  - If the nodes are connected?
    - Running time: $\boldsymbol{O}(|E|)$

- If $|V| = n$
- For directed graph
- $0 \leq |E| \leq n(n-1)$
- For undirected graph
- $0 \leq |E| \leq n(n-1)/2$

- If $|V| = v$ $\quad |E| = v(v-1) = v^2 - v = \boldsymbol{O}(v^2)$
- 10 $\quad = 90$
- 100 $\quad = 9900$
- 1000 $\quad = 999000$

- $\boldsymbol{O}(|E|)\ or\ \boldsymbol{O}(|\boldsymbol{v}| * |\boldsymbol{v}|) \rightarrow$ ***Costly***
- $\boldsymbol{O}(|\boldsymbol{v}|) \rightarrow$ OK

Adjacency Matrix



Vertex List

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

	0	1	2	3	4	5	6	7
0	0	1	1	1	0	0	0	0
1	1	0	0	0	1	1	0	0
2	1	0	0	0	0	0	1	0
3	1	0	0	0	0	0	0	1
4	0	1	0	0	0	0	0	1
5	0	1	0	0	0	0	0	1
6	0	0	1	0	0	0	0	1
7	0	0	0	1	1	1	1	0

$$A_{ij} = A_{ji}$$

$$A_{ij} = \begin{cases} 1, if\ \exists\ edge from\ i\ to\ j \\ 0, otherwise \end{cases}$$

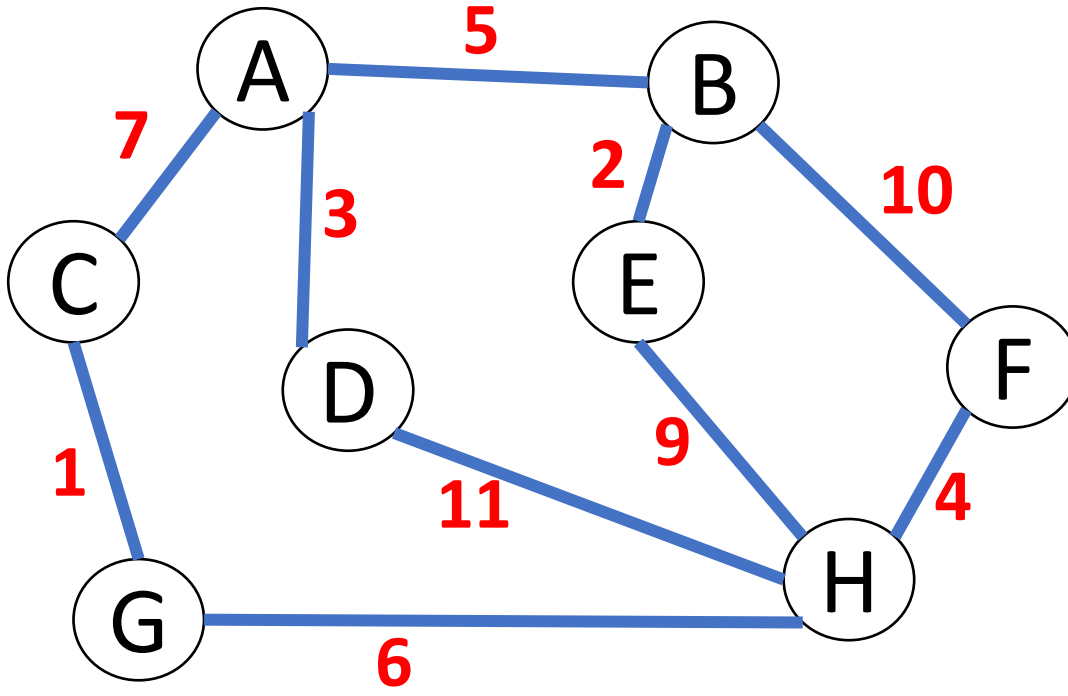
Vertex List

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

	0	1	2	3	4	5	6	7
0	0	1	1	1	0	0	0	0
1	1	0	0	0	1	1	0	0
2	1	0	0	0	0	0	1	0
3	1	0	0	0	0	0	0	1
4	0	1	0	0	0	0	0	1
5	0	1	0	0	0	0	0	1
6	0	0	1	0	0	0	0	1
7	0	0	0	1	1	1	1	0

Operation	Time Complexity
Finding adjacency nodes	$\boldsymbol{O}(v) + \boldsymbol{O}(v) = \boldsymbol{O}(v)$
Finding if two nodes are connected or not?	$\boldsymbol{O}(1) + \boldsymbol{O}(v) = \boldsymbol{O}(v)$

Adjacency Matrix



Vertex List

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

	0	1	2	3	4	5	6	7
0	∞	5	7	3	∞	∞	∞	∞
1	5	∞	∞	∞	2	10	∞	∞
2	7	∞	∞	∞	∞	∞	1	∞
3	3	∞	∞	∞	∞	∞	∞	11
4	∞	2	∞	∞	∞	∞	∞	9
5	∞	10	∞	∞	∞	∞	∞	4
6	∞	∞	1	∞	∞	∞	∞	6
7	∞	∞	∞	6	11	9	4	∞

$$A_{ij} = A_{ji}$$

$$A_{ij} = \begin{cases} 1, if\ \exists\ edge from\ i\ to\ j \\ 0, otherwise \end{cases}$$

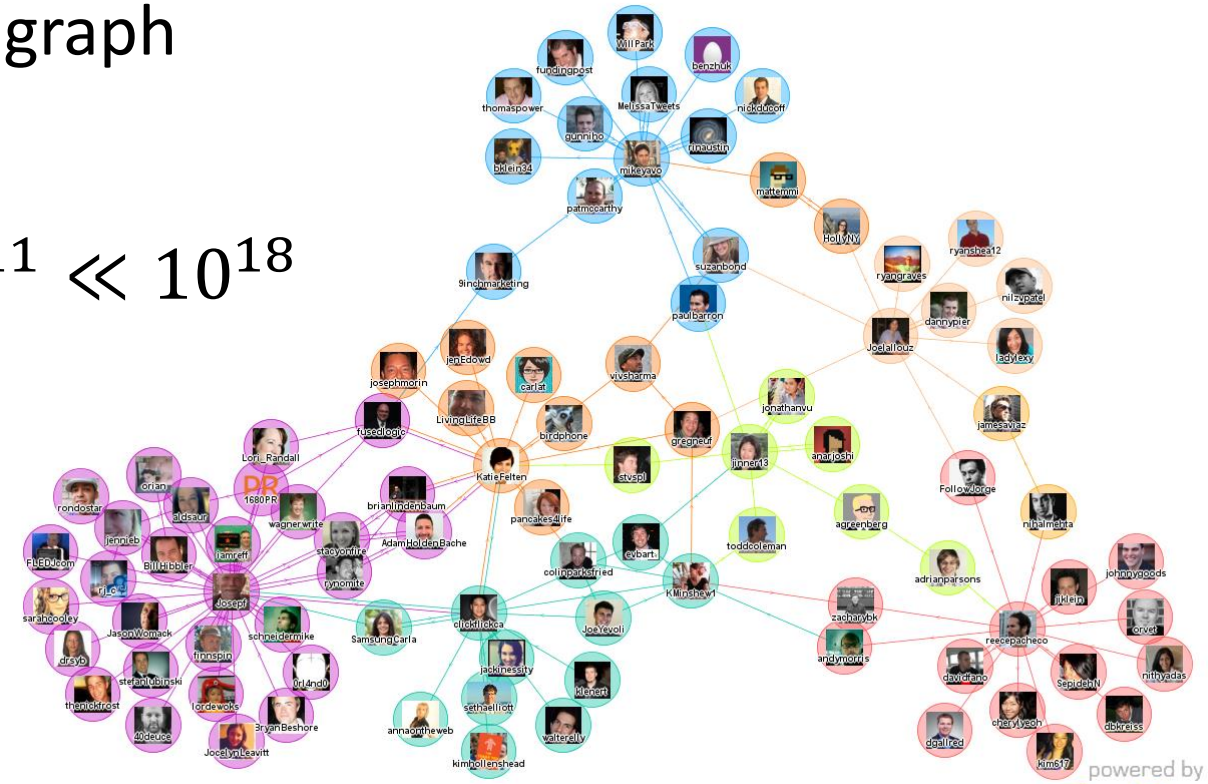
Vertex List

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

	0	1	2	3	4	5	6	7
0	0	1	1	1	0	0	0	0
1	1	0	0	0	1	1	0	0
2	1	0	0	0	0	0	1	0
3	1	0	0	0	0	0	0	1
4	0	1	0	0	0	0	0	1
5	0	1	0	0	0	0	0	1
6	0	0	1	0	0	0	0	1
7	0	0	0	1	1	1	1	0

$$\boldsymbol{O}(v^2)$$

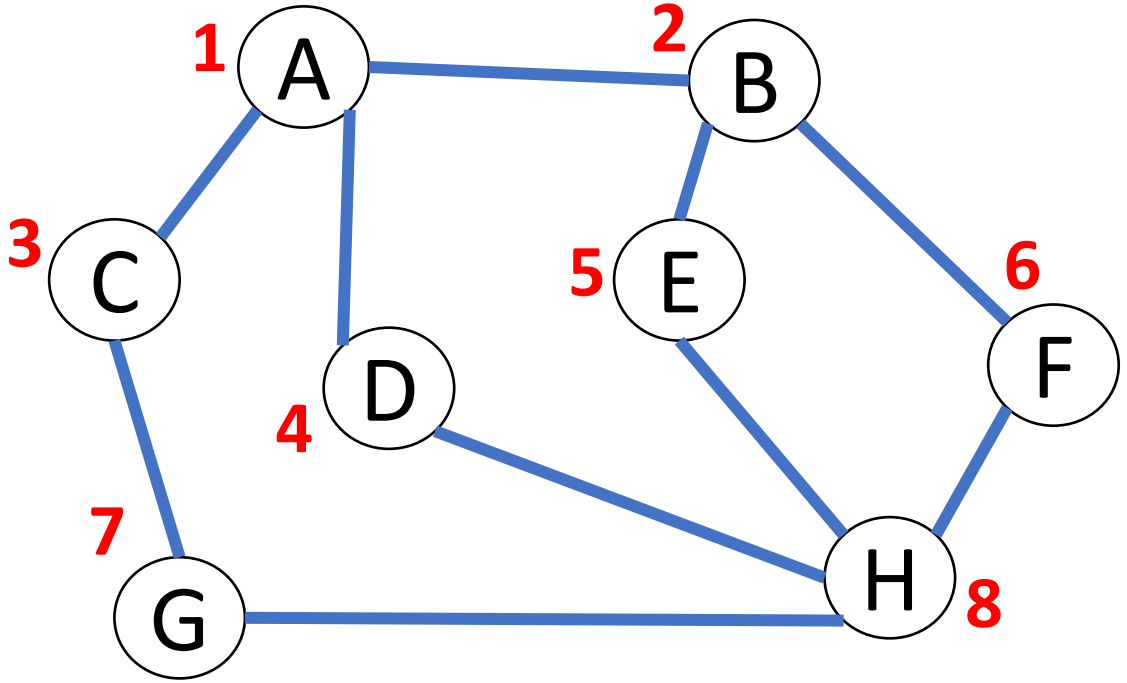
- If $|v| = 10^9$
- If every one is connected to everyone then $10^9 * 10^9 = 10^{18}$ connections in the graph
- If maximum friends allowed is 1000
- Then $|E| = \frac{10^9 * 10^3}{2} = \frac{10^{12}}{2} = 5 * 10^{11} \ll 10^{18}$
  - 10^{18} bytes $\approx$ 1000 PB
  - $5 * 10^{11}$ bytes $\approx$ 0.5 TB



Vertex List

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

	0	1	2	3	4	5	6	7
0	0	1	1	1	0	0	0	0
1	1	0	0	0	1	1	0	0
2	1	0	0	0	0	0	1	0
3	1	0	0	0	0	0	0	1
4	0	1	0	0	0	0	0	1
5	0	1	0	0	0	0	0	1
6	0	0	1	0	0	0	0	1
7	0	0	0	1	1	1	1	0



Connection of node A

0	1	2	3	4	5	6	7
0	1	1	1	0	0	0	0

0	1	2	3	4	5	6	7
0	1	1	1	0	0	0	0

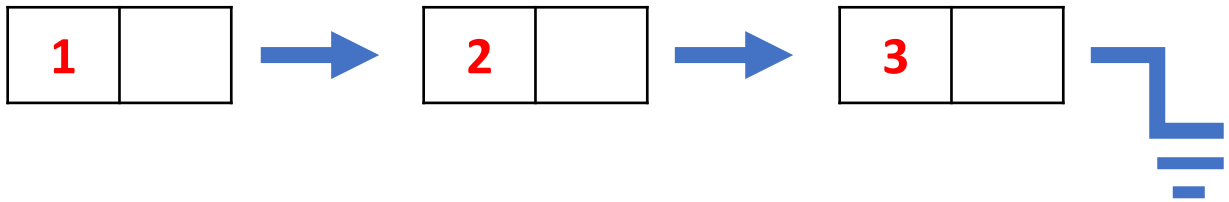
- For a social network graph with a billion users: Row size = 10^9
- If max. friends allowed is 1000
  - Number of 1s would be 1000 ≈ 1 KB (in case of Boolean value)
  - Number of 0s would be 10^9 - 1000 ≈ 1 GB (in case of Boolean value)

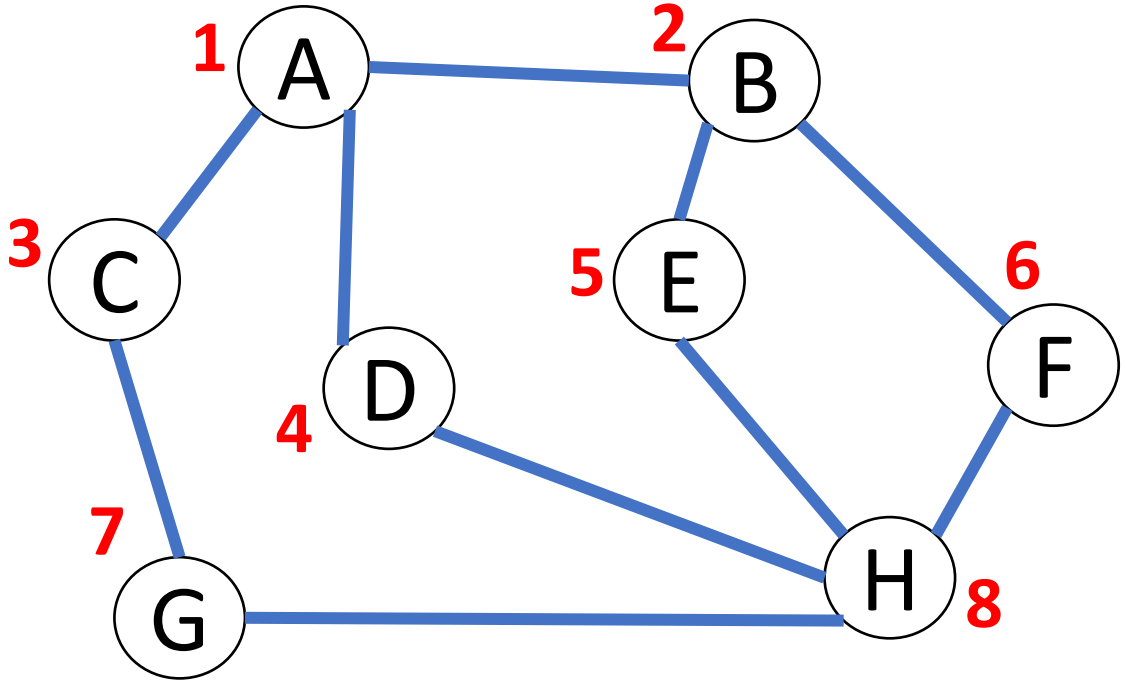
Connection of node A

0	1	2	3	4	5	6	7
0	1	1	1	0	0	0	0

$\{1,2,3\} or \{B, C, D\}$

0	1	2
1	2	3





Space = $O(v^2)$	Space = $O(e)$ In most graphs $e \ll v^2$

	0	1	2	3	4	5	6	7
0	0	1	1	1	0	0	0	0
1	1	0	0	0	1	1	0	0
2	1	0	0	0	0	0	1	0
3	1	0	0	0	0	0	0	1
4	0	1	0	0	0	0	0	1
5	0	1	0	0	0	0	0	1
6	0	0	1	0	0	0	0	1
7	0	0	0	1	1	1	1	0

0	1	2	3	
1	0	4	5	
2	0	6		
3	0	7		
4	1	7		
5	1	7		
6	2	7		
7	3	6	4	5

```
int *A[8];
A[0] = new int[3];
A[1] = new int[3];
A[2] = new int[2];
A[3] = new int[2];
A[4] = new int[2];
A[5] = new int[2];
A[6] = new int[2];
A[7] = new int[4];
```

	Adjacency Matrix	Linked List
Finding if two nodes are connected?	$O(1)$	$O(v)$ or $O(\log(v))$
Finding all adjacent nodes	$O(v)$	$O(v)$